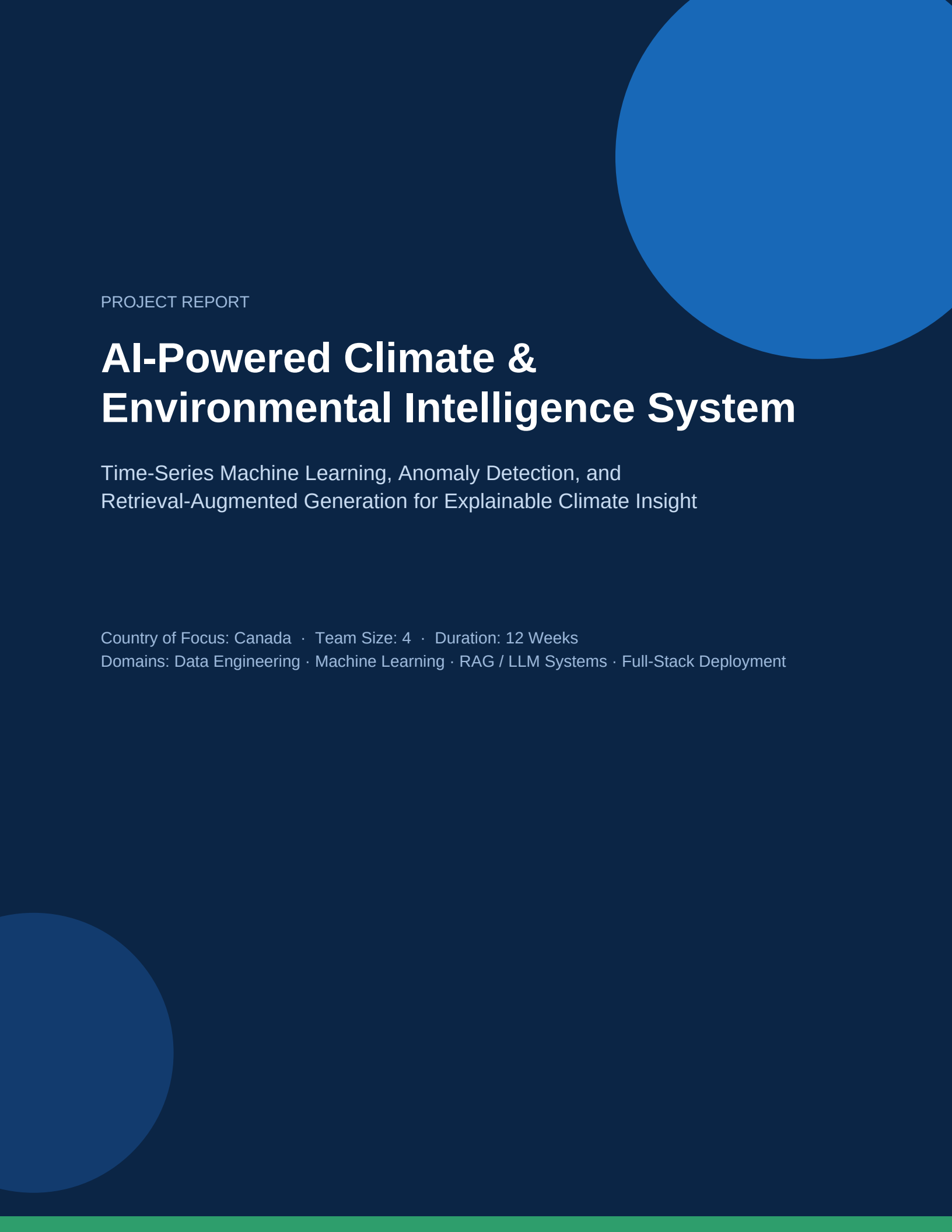


The background features a dark blue gradient with two large, semi-transparent blue circles. One circle is in the top right corner, and the other is in the bottom left corner.

PROJECT REPORT

AI-Powered Climate & Environmental Intelligence System

Time-Series Machine Learning, Anomaly Detection, and
Retrieval-Augmented Generation for Explainable Climate Insight

Country of Focus: Canada · Team Size: 4 · Duration: 12 Weeks

Domains: Data Engineering · Machine Learning · RAG / LLM Systems · Full-Stack Deployment

Table of Contents

1.	Project Overview & Motivation
2.	Problem Statement
3.	System Concept — What Makes It More Than a Weather App
4.	Data Layer — Sources & Variables
5.	Machine Learning Layer
6.	Retrieval-Augmented Generation (RAG) Layer
7.	End-to-End Architecture & Pipeline Flow
8.	Database Architecture
9.	Example User Interaction Walkthrough
10.	Technology Stack
11.	Team Structure & Role Breakdown
12.	Twelve-Week Roadmap
13.	Evaluation Metrics & Success Criteria
14.	Risks & Mitigations

1. Project Overview & Motivation

This project builds a country-scoped Climate & Environmental Intelligence System that goes beyond conventional weather applications. Rather than only displaying forecasts, the system learns long-term climate patterns, detects statistically unusual events, classifies those events into named phenomena (heatwaves, cold waves, heavy rainfall, drought trends, high-wind events), and then explains each event using evidence retrieved from authoritative climate research and government reports.

Scoping the project to a single country — rather than attempting a shallow "global climate AI" — produces a far stronger and more defensible result. A focused scope allows the team to build cleaner seasonal baselines, richer feature engineering, and a knowledge base tailored to a specific region's climate literature. Canada is used as the reference implementation throughout this report because of its excellent public data availability (Environment and Climate Change Canada, NASA, NOAA), relatively clean seasonal patterns, and well-documented climate anomalies such as heatwaves, cold waves, and wildfire seasons — though the same architecture generalizes directly to Norway, Finland, Sweden, or any comparable non-equatorial country.

Why avoid equatorial regions?

Equatorial and tropical climates tend to have high day-to-day variability and less distinct seasonal structure, which makes historical baselines noisier and anomaly detection less reliable for a project of this scope. Mid-to-high-latitude countries offer cleaner seasonality, making trends and anomalies easier to model and easier to validate.

2. Problem Statement

Current weather apps answer a narrow question: *what is the weather right now, or what will it be soon?* They rarely explain the *why* behind an event, its historical rarity, or its scientific significance.

This system is designed to answer a deeper set of questions:

- Why did this weather anomaly happen?
- Is this event unusual compared to historical records for this location and date?
- What longer-term trend is developing?
- What do scientific studies and government reports say about this type of event?
- What should policymakers, researchers, or the public pay attention to going forward?

It answers these questions by combining API-collected climate data, supervised and unsupervised machine learning, time-series forecasting, anomaly detection, retrieval-augmented generation over climate research, and LLM-generated, evidence-backed explanations.

3. System Concept — What Makes It More Than a Weather App

The core differentiator is what can be called the **Climate Intelligence Layer**: a stack of models and retrieval systems sitting on top of raw climate data. Instead of simply displaying a forecast, the system performs six distinct functions:

- **Learns** historical seasonal and regional patterns using machine learning.

- **Detects** unusual events through statistical and ML-based anomaly detection.
- **Classifies** environmental events into named categories (heatwave, cold wave, heavy rainfall, drought trend, high-wind event).
- **Retrieves** scientific evidence from climate reports (IPCC, Environment Canada, NOAA, peer-reviewed research) using RAG.
- **Generates** explainable reports that cite their sources rather than relying on an LLM's unverified internal knowledge.
- **Visualizes** trends, deviation magnitude, and model confidence, making predictions interpretable rather than opaque.

This combination of time-series ML, anomaly detection, RAG, and explainable AI gives the project genuine research and engineering depth while remaining realistic for a four-person team to deliver over a three-month timeline.

4. Data Layer — Sources & Variables

A robust system needs more than temperature alone. The data layer collects multiple correlated climate variables so that downstream models have enough signal to distinguish, for example, a genuine heatwave from a single warm reading.

4.1 Variables Collected

Category	Examples
Temperature	Maximum, minimum, and average daily temperature
Rainfall	Daily precipitation totals
Humidity	Relative humidity
Wind	Speed and direction
Pressure	Atmospheric pressure
Air Quality	AQI, PM2.5 (optional enrichment)
Snow	Snowfall — a distinct advantage of the Canada dataset

4.2 Recommended APIs

Source	What it provides
Environment and Climate Change Canada (ECCC)	Historical weather data, forecasts, and official climate observations
NASA POWER API	Temperature, solar radiation, humidity, and long historical coverage
NOAA	Climate datasets and anomaly records
Open-Meteo	Free, developer-friendly weather API, useful for rapid prototyping

5. Machine Learning Layer

Many student and hackathon projects only wrap an LLM around static data and call it "AI." This system avoids that shortcut by actually training three distinct models, each with a clear input, output, and evaluation metric.

5.1 Model 1 — Weather Trend Forecasting

Purpose: predict near-term future values of key climate variables (e.g., temperature, precipitation) from historical time series.

Candidate algorithms: XGBoost, Random Forest, Prophet, and — as an optional deep learning extension — LSTM networks for sequential dependencies.

This model gives the project a genuine forecasting component, evaluated with standard regression metrics rather than presented as a black box.

5.2 Model 2 — Climate Anomaly Detection

Purpose: answer the question, "Is today's reading unusual for this location and date?" This is arguably the most interesting model in the system, because it converts raw numbers into a flagged event.

Worked example:

	Date	Temperature
Historical average	July 10	22°C
Current reading	July 10	31°C

Deviation = 31°C – 22°C = **9°C above baseline** → the system flags: *"Heat anomaly detected."*

Candidate algorithms: Isolation Forest, Local Outlier Factor, One-Class SVM.

5.3 Model 3 — Environmental Event Classification

Purpose: once an anomaly is detected, classify it into a named category so the system knows what to explain and what evidence to retrieve.

Candidate classes: heatwave, cold wave, heavy rainfall, drought trend, high-wind event.

Input features: temperature, humidity, rainfall, pressure, wind.

The classified event label is the bridge between the ML layer and the RAG layer — it becomes the query that determines which climate research is retrieved next.

6. Retrieval-Augmented Generation (RAG) Layer

This is where the project moves from "machine learning demo" to a genuinely useful decision-support tool. Suppose the ML layer detects a heatwave. Rather than letting a language model generate an explanation purely from its internal (and possibly outdated or hallucinated) knowledge, the system retrieves real evidence from a curated knowledge base first, and only then asks the LLM to write an explanation grounded in that evidence.

6.1 Knowledge Base Sources

- IPCC assessment and synthesis reports
- Environment Canada climate reports
- NOAA studies
- Peer-reviewed climate research papers
- Government climate advisories

6.2 Retrieval Pipeline

- Collect and download source PDFs/reports into a document store.
- Parse and chunk documents into retrieval-sized passages.
- Generate embeddings for each chunk using a sentence-transformer model.
- Index embeddings in a vector database.
- At query time, embed the detected event + location + date as a query.
- Retrieve the top-k most relevant passages.
- Pass retrieved passages, along with the ML outputs, to the LLM as grounding context.
- LLM generates a cited, evidence-backed explanation.

With this pipeline, the LLM's output changes from an ungrounded guess to a statement such as: *"This event resembles previous Canadian heatwaves documented in [source], with citations."* The explanation is not generated from thin air — it is grounded in retrieved documents, which is the key trust-building property of RAG systems.

7. End-to-End Architecture & Pipeline Flow

The system flows through five conceptual stages, from raw data to a cited, user-facing explanation:

① Collect

Scheduled API pulls from ECCC, NASA POWER, NOAA, and Open-Meteo populate the PostgreSQL climate database.

② Engineer

Raw readings are cleaned, missing values imputed, and features such as day-of-year, 7-day and 30-day rolling averages, and historical baselines are computed.

③ **Model**

The forecasting model predicts near-term values; the anomaly detector flags unusual readings; the classifier labels the anomaly type.

④ **Retrieve**

The event label, location, and date form a query embedded and matched against the vector database of climate reports.

⑤ **Explain**

The LLM combines ML outputs and retrieved evidence into a cited, human-readable explanation, served through the FastAPI backend to the dashboard.

8. Database Architecture

The system uses three databases, each optimized for a different kind of data:

Database	Purpose
PostgreSQL	Structured storage for raw and engineered climate data (temperature, rainfall, humidity, wind, pressure, etc.)
Qdrant / Chroma	Vector database storing embeddings of climate report chunks for semantic retrieval
Redis (optional)	Caching layer for frequently requested queries and dashboard responses, improving latency

9. Example User Interaction Walkthrough

A user opens the dashboard and selects a province — **Alberta** — and a date — **July 15**. The Climate Intelligence Dashboard responds with:

Field	Value
Current temperature	31°C
Historical average	22°C
Classification	Heat anomaly
Deviation from baseline	+9°C (Severity: High)
Model confidence	91%
AI explanation	"The anomaly exceeds the historical average for this date and resembles documented Canadian heatwave patterns from previous years."
Sources used	IPCC report; Environment Canada study

The key design principle on display here: every explanation shown to the user is traceable to both a quantitative model output (deviation, confidence) and a cited source document — nothing is asserted without evidence.

10. Technology Stack

Layer	Technology
Language	Python
Machine Learning	Scikit-learn
Deep Learning (optional)	PyTorch
Data Processing	Pandas
API Access	Requests

Layer	Technology
Backend	FastAPI
Relational Database	PostgreSQL
Vector Database	Qdrant
RAG Orchestration	LangChain
Embeddings	Sentence Transformers
LLM	Llama 3.1 / Mistral
Deployment	Docker Compose

11. Team Structure & Role Breakdown

The project is designed for a four-person team, with clearly separated responsibilities to minimize overlap and merge conflicts.

Person 1 — Data Pipeline

Tasks:

- API integration
- Scheduled data collection
- Data cleaning & missing-value handling
- PostgreSQL schema and storage

Deliverable: A reliable, continuously updated climate database.

Tools: Python, Pandas, Requests, PostgreSQL

Person 2 — ML Engineer

Tasks:

- Forecasting model
- Anomaly detection model
- Event classification model
- Model evaluation

Deliverable: Trained models, evaluation metrics, and prediction outputs.

Tools: Scikit-learn, XGBoost/Prophet, PyTorch (optional)

Person 3 — RAG Engineer

Tasks:

- Collect climate reports
- PDF processing & chunking
- Generate embeddings
- Vector DB indexing & retrieval evaluation

Deliverable: A working retrieval system returning relevant, grounded evidence.

Tools: LangChain / LlamaIndex, Sentence Transformers, Qdrant

Person 4 — Backend & DevOps

Tasks:

- FastAPI service layer
- Dashboard/frontend

- Authentication
- Docker packaging, deployment, and system integration

Deliverable: A deployed, end-to-end working application.

Tools: FastAPI, Docker, React (or a simple frontend)

Suggested evaluation metrics for the ML role:

- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)
- Precision
- Recall

12. Twelve-Week Roadmap

Phase 1 — Foundation (Weeks 1–2)

Goal: collect reliable data.

- Choose the target country.
- Choose and register for APIs.
- Build the data collector.
- Store data in PostgreSQL.
- Exploratory data analysis.

Deliverable: a working climate database.

Phase 2 — Data Engineering (Weeks 3–4)

- Handle missing values.
- Feature engineering.
- Compute moving averages.
- Build seasonal features.
- Compute historical baselines.

Feature	Why it matters
Day of year	Captures seasonality
7-day average	Captures short-term trend
30-day average	Establishes a baseline
Temperature deviation	Feeds directly into anomaly detection

Deliverable: a model-ready dataset.

Phase 3 — Machine Learning (Weeks 5–6)

- Train the forecasting model.
- Train the anomaly detection model.
- Train the event classification model.

Deliverable: trained models with logged evaluation metrics.

Phase 4 — RAG (Weeks 7–8)

- Download and curate climate reports.
- Build the vector database.
- Test and tune retrieval quality.
- Connect the LLM to the retrieval pipeline.

Example query: *"Why is Alberta experiencing unusual heat?"* — the system retrieves passages from IPCC, Canadian climate reports, and NOAA studies.

Deliverable: grounded, cited answers to natural-language climate questions.

Phase 5 — Integration (Weeks 9–10)

Connect the data pipeline, ML layer, RAG layer, and backend into a single end-to-end system so that a user query flows from the dashboard through every component and back as a cited explanation.

Deliverable: a fully connected, end-to-end system.

Phase 6 — Polish (Weeks 11–12)

- UI improvements.
- Docker Compose packaging for one-command deployment.
- Testing.
- Documentation.
- Viva / demo preparation.

Deliverables: final project, written report, and demo video.

13. Evaluation Metrics & Success Criteria

Component	Metric(s)	Target Signal
Forecasting model	MAE, RMSE	Low error relative to naive baseline (e.g., yesterday's value)
Anomaly detector	Precision, Recall	Flags true anomalies without excessive false positives
Event classifier	Accuracy, F1-score	Correctly labels the anomaly type
Retrieval (RAG)	Top-k relevance, hit rate	Retrieved passages are actually relevant to the query
End-to-end system	Latency, uptime, user testing	Dashboard responds promptly with a coherent, cited explanation

14. Risks & Mitigations

Risk	Mitigation
API rate limits or downtime	Cache pulled data locally; stagger scheduled collection jobs across sources
Sparse historical data for rare events	Combine multiple sources (ECCC + NASA + NOAA) to widen the historical window
LLM hallucination in explanations	Enforce RAG grounding — the LLM must cite retrieved passages, and prompts should refuse to answer without supporting evidence
Team merge conflicts across four workstreams	Strict module boundaries (data / ML / RAG / backend) with a shared schema agreed in Phase 1
Scope creep before the deadline	Treat Phase 6 as polish-only; feature-freeze after Phase 5 integration

Summary: By combining time-series forecasting, anomaly detection, event classification, retrieval-augmented generation, and an explainable, cited presentation layer, this project moves well beyond a typical weather-display app into a genuine climate intelligence system — one that is scoped realistically for a four-person team to design, build, and demo within twelve weeks.